

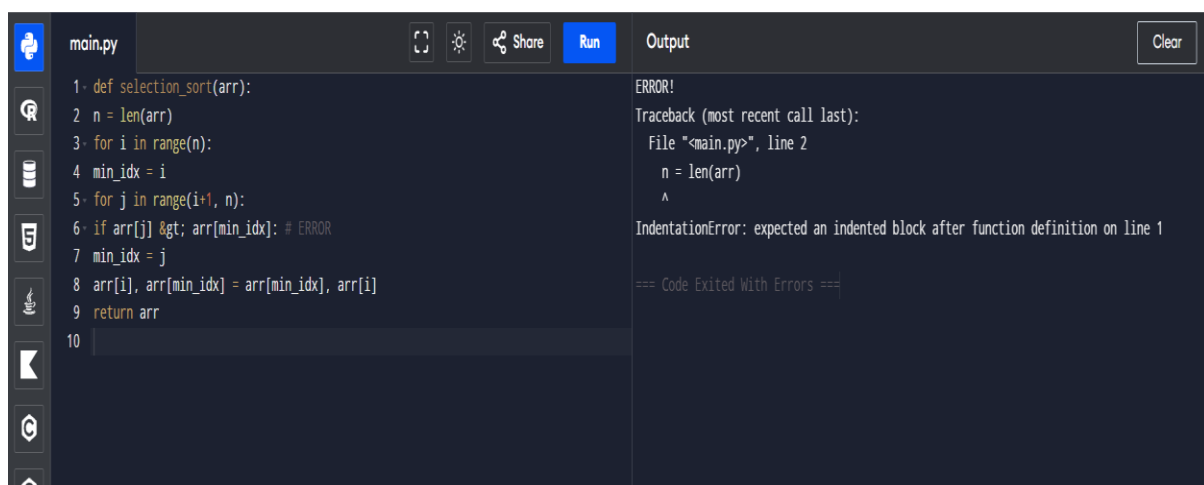
NAME: Aksharaa.B

REG NO.:192311278

COURSE CODE:CSA0603-DAA

Q1. Debugging Selection Sort Code The following Selection Sort code contains logical errors and produces incorrect output for some inputs. Carefully analyze the code, identify all errors in comparison and swapping logic, and explain their root causes. Apply systematic debugging to correct the algorithm step-by-step. Optimize the implementation by reducing unnecessary comparisons if possible. Analyze the corrected algorithm in terms of time complexity. Rewrite the final optimized code with proper structure and comments.

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = I  
        for j in range(i+1, n):  
            if arr[j] > arr[min_idx]: # ERROR  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr
```



The screenshot shows a code editor with a file named 'main.py'. The code is a Python function 'selection_sort' that takes an array 'arr' and returns it. The function has several logical errors and one syntax error. The syntax error is an 'IndentationError' on line 1, where the function definition is not properly indented. The logical errors are in the comparison and swapping logic. The comparison 'if arr[j] > arr[min_idx]' is incorrect for finding the minimum element. The swapping logic 'arr[i], arr[min_idx] = arr[min_idx], arr[i]' is also incorrect. The output pane shows the error message: 'ERROR! Traceback (most recent call last): File "<main.py>", line 2 n = len(arr) ^ IndentationError: expected an indented block after function definition on line 1'. Below the error message, it says '=== Code Exited With Errors ==='.

```
1 def selection_sort(arr):  
2     n = len(arr)  
3     for i in range(n):  
4         min_idx = i  
5         for j in range(i+1, n):  
6             if arr[j] > arr[min_idx]: # ERROR  
7                 min_idx = j  
8         arr[i], arr[min_idx] = arr[min_idx], arr[i]  
9     return arr  
10
```

ERROR!
Traceback (most recent call last):
File "<main.py>", line 2
 n = len(arr)
 ^
IndentationError: expected an indented block after function definition on line 1

=== Code Exited With Errors ===

SOLUTION:

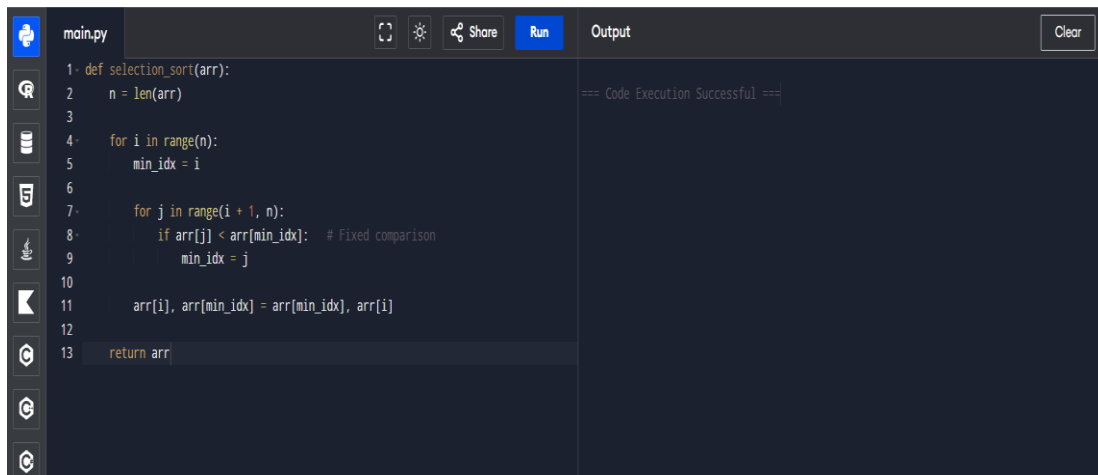
Step 1: Understand How Selection Sort Works

Selection Sort repeatedly:

1. Finds the smallest element in the unsorted portion of the array.
2. Swaps it with the first unsorted element.
3. Expands the sorted portion by one position.

For ascending order:

- `min_idx` should always store the index of the minimum element.
- Therefore, comparisons should use `<`, not `>`.



```
main.py
1- def selection_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         min_idx = i
6-
7-         for j in range(i + 1, n):
8-             if arr[j] < arr[min_idx]: # Fixed comparison
9-                 min_idx = j
10-
11-         arr[i], arr[min_idx] = arr[min_idx], arr[i]
12-
13-     return arr
```

Output

=== Code Execution Successful ===

Step 2: Identify the Errors

- Error 1: Incorrect Comparison Operator
- Problem: `if arr[j] > arr[min_idx]:`
This updates `min_idx` whenever a larger element is found.

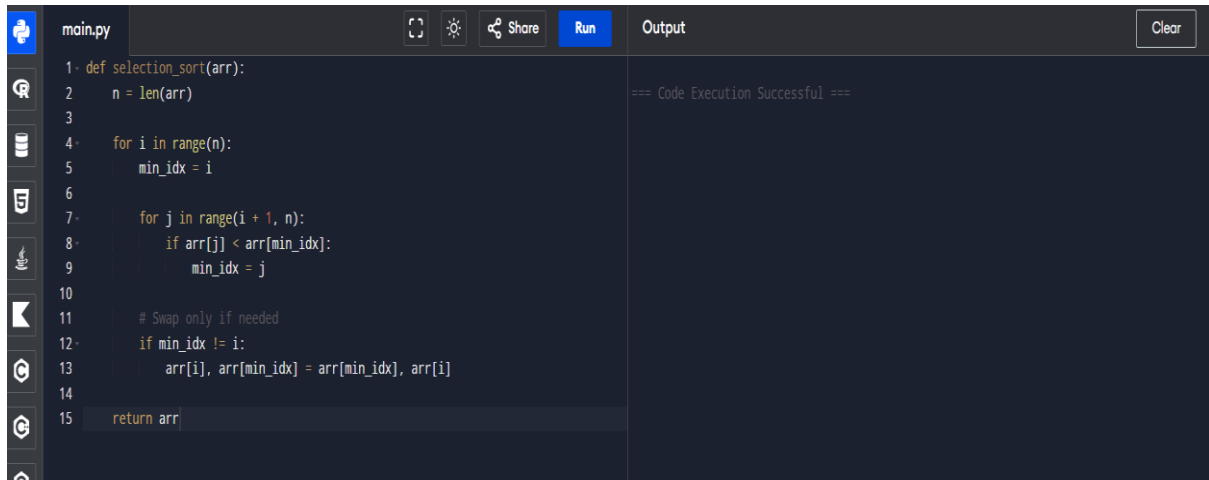
Root Cause

- The variable name `min_idx` implies that we are searching for the minimum element, but the condition actually searches for the maximum element.

Error 2: Unnecessary Swaps

- Problem: `arr[i], arr[min_idx] = arr[min_idx], arr[i]`

- A swap occurs even when: `min_idx == i`
- The algorithm performs redundant operations when the current element is already the minimum.
- Problem: if `min_idx != i`:
- Sol: `arr[i], arr[min_idx] = arr[min_idx], arr[i]`



```

main.py
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         min_idx = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_idx]:
9                 min_idx = j
10
11         # Swap only if needed
12         if min_idx != i:
13             arr[i], arr[min_idx] = arr[min_idx], arr[i]
14
15     return arr

```

Output: === Code Execution Successful ===

Step 3: Optimization

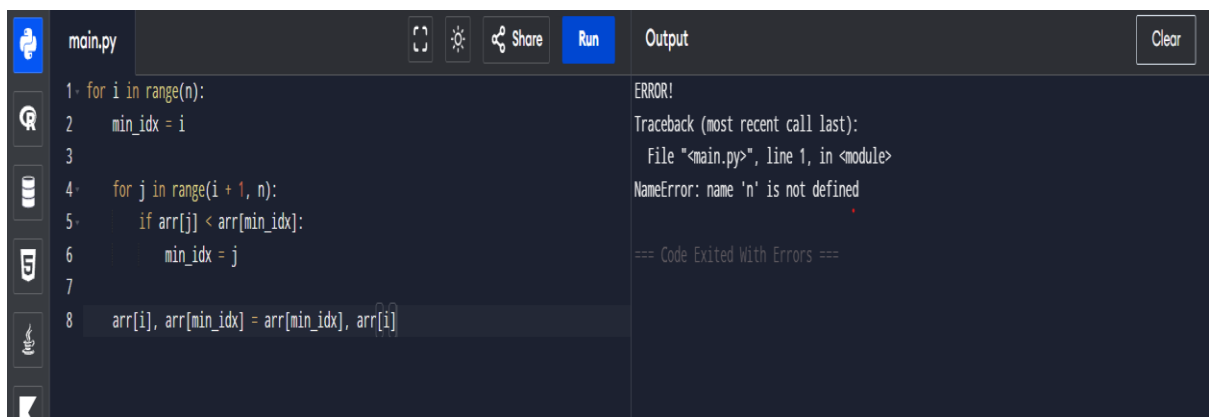
The outer loop does not need to run for the last element because after sorting the first $n-1$ positions, the final element is automatically in the correct position.

Instead of

`for i in range(n):`

Use

`for i in range(n - 1):`



```

main.py
1 for i in range(n):
2     min_idx = i
3
4     for j in range(i + 1, n):
5         if arr[j] < arr[min_idx]:
6             min_idx = j
7
8     arr[i], arr[min_idx] = arr[min_idx], arr[i]

```

Output: ERROR!
Traceback (most recent call last):
File "<main.py>", line 1, in <module>
NameError: name 'n' is not defined
=== Code Exited With Errors ===

Before optimization

×

⏮ Hide code

```
if min_idx != i:
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Console

Run started
Initializing environment
Installing packages
Running code
NameError: name 'min_idx' is not defined
Run completed in 1802.199999988079ms

After optimization

Final Optimized Code

```
def selection_sort(arr)
```

Sorts a list in ascending order using Selection Sort.

Parameters:

arr (list): List of comparable elements

Returns:

list: Sorted list

```
"""
```

```
n = len(arr)
```

```
# Traverse through all positions except the last
```

```
for i in range(n - 1):
```

```
    # Assume the current position contains the minimum
```

```
    min_idx = i
```

```
    # Find the smallest element in the unsorted portion
```

```
    for j in range(i + 1, n):
```

```
        if arr[j] < arr[min_idx]:
```

```
            min_idx = j
```

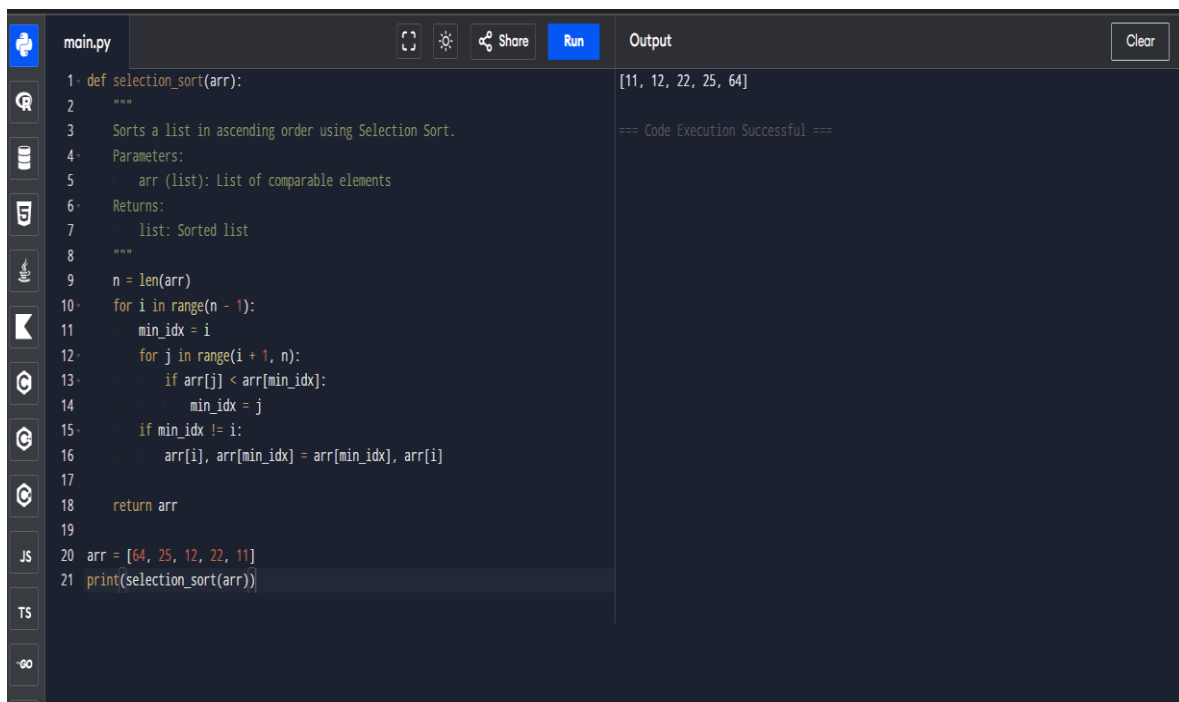
```

        # Swap only if a smaller element was found
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Example usage
arr = [64, 25, 12, 22, 11]
print(selection_sort(arr))

```



The screenshot shows a code editor with a dark theme. On the left, a sidebar contains icons for file explorer, search, and other IDE features. The main editor area displays a Python file named 'main.py' with the following code:

```

1 def selection_sort(arr):
2     """
3     Sorts a list in ascending order using Selection Sort.
4     Parameters:
5     arr (list): List of comparable elements
6     Returns:
7     list: Sorted list
8     """
9     n = len(arr)
10    for i in range(n - 1):
11        min_idx = i
12        for j in range(i + 1, n):
13            if arr[j] < arr[min_idx]:
14                min_idx = j
15        if min_idx != i:
16            arr[i], arr[min_idx] = arr[min_idx], arr[i]
17
18    return arr
19
20 arr = [64, 25, 12, 22, 11]
21 print(selection_sort(arr))

```

On the right side of the editor, there is an 'Output' panel. It shows the result of the code execution: '[11, 12, 22, 25, 64]'. Below the output, it says '=== Code Execution Successful ==='. There is a 'Clear' button next to the output panel.

The corrected Selection Sort algorithm successfully sorts the array in **ascending order** by repeatedly finding the smallest element from the unsorted portion of the list and placing it in its correct position. During each pass, the algorithm maintains `min_idx` as the index of the minimum element and swaps it with the current position only when necessary.

Thus, the output is a fully sorted array in ascending order. The optimization of avoiding unnecessary swaps improves efficiency by reducing write operations, although the number of comparisons remains the same. Overall, the algorithm

has a time complexity of $O(n^2)$ and a space complexity of $O(1)$, making it suitable for small datasets but less efficient than advanced sorting algorithms for large inputs.